

优先级位图算法解析

uC/OS-II 中的优先级位图算法是该实时操作系统的核心精髓之一。在实时操作系统中，系统必须在极短且确定的时间内（即 $O(1)$ 时间复杂度）从所有就绪任务中找到优先级最高的任务来执行。位图算法正是采用“空间换时间”的策略，通过巧妙的位运算和查表法完美实现了这一目标。

结合你提供的资料，我为你梳理一下这个算法的运行逻辑：

1. 核心数据结构与优先级拆解

uC/OS-II 最多支持 64 个任务优先级，范围是 0~63，**数字越小优先级越高**（0 级为最高）。

这 64 个优先级被安置在一个 8×8 的矩阵中，由两个核心变量来管理：

- **OSRdyTbl[8]（优先级就绪表）**：这是一个包含 8 个元素的数组，每个元素也是 8 位变量。它具体记录了每一行中哪一列的任务已经就绪。

当我们拥有一个优先级数字 **prio**（只有低 6 位有效）时，系统会将其拆分为高 3 位和低 3 位来进行矩阵定位：

- **高 3 位 (Y)**：对应矩阵的行（组）。`OSTCBY = prio >> 3` 获取高 3 位，指示任务在 **OSRdyGrp** 中的具体位置。
- **低 3 位 (X)**：对应矩阵的列（位）。`OSTCBX = prio & 0x07` 获取低 3 位，指示任务在 **OSRdyTbl[]** 中的具体位置。

2. 映射表 **OSMapTbl[]** 与任务就绪（入列）

为了快速将计算出的 X 和 Y 转换为对应的二进制位（即把某一位强行置 1），系统引入了映射表 **OSMapTbl[]**。

- 这个表很简单：下标 0 对应 **00000001**，下标 1 对应 **00000010**，以此类推。
- 通过 `OSTCBBitY = OSMapTbl[OSTCBY]` 和 `OSTCBBitX = OSMapTbl[OSTCBX]`，系统获得了操作行和列的位掩码。

当一个任务就绪时，如何打上标记？

两行极其高效的 C 代码即可完成：

1. `OSRdyGrp |= OSMapTbl[prio >> 3];` （将对应组的标志位置 1）
2. `OSRdyTbl[prio >> 3] |= OSMapTbl[prio & 0x07];` （将对应表中的具体任务标志位置 1）

3. 判定表 `OSUnMapTbl[]` 与寻找最高优先级（出列）

当有多个任务同时就绪时，`OSRdyGrp` 和 `OSRdyTbl[]` 中会有多个位被置 1。此时，系统需要找到所有被置 1 的位中，**位置最低的那个**（因为数字越小优先级越高）。

如果用循环去逐位判断，耗时会不确定，这就犯了实时操作系统的兵家大忌。因此，uC/OS-II 引入了著名的优先级判定表 `OSUnMapTbl[256]`。

- **`OSUnMapTbl[]` 的魔法：**这是一个预先计算好的拥有 256 个元素的数组，它列举了 0x00 到 0xFF 之间每一个 8 位二进制数中**最低位的 1 所在的位置**（0~7）。例如，`0x0a`（二进制 `00001010`）中最低的 1 在第 1 位，所以 `OSUnMapTbl[0x0a] == 1`。

查找最高优先级的过程如下：

1. **找行 (Y)：** $y = \text{OSUnMapTbl}[\text{OSRdyGrp}]$ ；\$ 获取最高优先级所在的组（D5、D4、D3 位）。
2. **找列 (X)：** $x = \text{OSUnMapTbl}[\text{OSRdyTbl}[y]]$ ；\$ 在该组中寻找最高优先级的具体位置（D2、D1、D0 位）。
3. **合成优先级：** $\text{\$prio} = (y \gg 3) + x$ ；\$（也可以理解为 $\text{\$prio} = y \times 8 + x$ ），由此瞬间得出当前就绪的最高优先级。

总结来说，`OSMapTbl` 是用于“打包”把任务挂载到就绪表中，而 `OSUnMapTbl` 则是用于“解包”光速查找到最高优先级的任务。这种无需遍历的算法设计，极大保证了 uC/OS-II 任务调度的稳定性和高实时性。